UNIVERSIDAD — INGENIERÍA DE SISTEMAS / INFORMÁTICA

CÁTEDRA DE PROGRAMACIÓN GRÁFICA

SIMULADOR DE CONDUCCIÓN URBANA 3D

Implementación en Java y OpenGL 3.3 Core Profile mediante LWJGL

Asignatura:	Programación Gráfica
Entorno Gráfico:	Java 17, LWJGL 3, OpenGL 3.3 Core Profile, GLSL
Matemáticas y Vectores:	JOML 1.10.5 (matrices MVP, vectores 3D y 4D)
Escenario y Tamaño:	Cuadrícula urbana de 21 × 21 celdas (210 × 210 metros)
Mecánicas Clave:	Conducción física, ciclo día/noche con farolas automáticas, objetos destruibles, tráfico NPC y misiones de entrega
Gestor de Proyecto:	Apache Maven (compilación y pruebas unitarias con JUnit)
Fecha:	Octubre 2026

1. Diseño del Escenario y la Ciudad

Para armar el mundo del juego creé una ciudad sobre una cuadrícula regular de 21 × 21 celdas. Como cada celda mide 10 metros de lado, el mapa completo abarca 210 × 210 metros en espacio 3D. El objetivo fue armar un circuito urbano variado donde se pueda manejar a velocidad sin callejones sin salida y con puntos de referencia visual claros para no perderse.

Zonas y Sectores del Mapa

- Downtown (Centro Financiero): Rascacielos de hasta 42 metros de altura construidos con cajas modulares. Durante la noche, las ventanas activan un componente emisivo en el shader para simular oficinas iluminadas.
- Barrio Residencial: Casas y edificios de 2 a 4 pisos con techos a dos aguas y variaciones de color cálido.
- Catedral Central: Ubicada en el centro de la ciudad. Tiene pórtico con columnas, dos torres frontales y una cúpula octogonal. Funciona como punto de referencia visual principal desde casi cualquier calle.
- Parques Urbanos: Cuatro manzanas verdes situadas en puntos estratégicos, con césped elevado y caminerías de acera.
- Río y Puentes: Un cauce de agua de 30 metros de ancho que atraviesa la ciudad de norte a sur. Se cruza mediante 3 puentes de concreto con barandas protectoras, además de sectores con rampas y puente levadizo para probar saltos con el auto.
- Muro Perimetral: Las orillas externas del mapa están cerradas con muros continuos de contención para evitar que el auto salga disparado al vacío si se descontrola a alta velocidad.

Distribución de Árboles: Calzadas Libres para Conducir

Una decisión práctica que tomé al programar el generador de la ciudad fue no colocar ningún árbol sobre las pistas ni en las aceras de cruce. En pruebas iniciales, los árboles en las esquinas trababan al auto al doblar rápido. Por eso, todos los árboles quedaron restringidos al interior de los 4 parques urbanos. De esta manera, las calles están 100% limpias para correr, y quien quiera chocar contra la vegetación y ver cómo se rompe simplemente entra al parque.

Alumbrado Público Inteligente

Distribuí 34 farolas de 13 metros de altura en esquinas clave y en los accesos a los puentes, con un brazo que vuela 3.2 metros hacia el centro de la pista. Tienen lógica de fotocélula automática: durante el día permanecen apagadas para no sobrecargar el cálculo de luces innecesariamente; al activar el modo noche con la tecla N, se encienden solas proyectando conos de luz cálida sobre el asfalto.

2. Controles del Simulador

La captura de entradas se realiza en tiempo real mediante los callbacks de eventos de teclado de GLFW. Las teclas están organizadas para poder conducir cómodamente con una mano (WASD o flechas) y activar funciones con la otra:

Tecla	Acción	Comportamiento en el Simulador
W / Flecha Arriba	Acelerar	Aplica fuerza motriz hacia adelante con una curva progresiva de aceleración.
S / Flecha Abajo	Freno / Reversa	Aplica fricción a las ruedas; al detenerse por completo activa la marcha atrás.
A / D	Girar Izq. / Der.	Modifica el ángulo de dirección. A mayor velocidad, el giro se suaviza para evitar trompos.
Barra Espaciadora	Freno de Mano	Traba las ruedas traseras para detenciones en seco y derrapes en las curvas.
Shift Izquierdo	Nitro / Turbo	Multiplica la fuerza de aceleración y aumenta la velocidad tope mientras se mantenga presionado.
C	Cambiar Cámara	Alterna entre 3ª persona (persecución), 1ª persona (capó) y vista cenital aérea.
N	Alternar Día / Noche	Cambia la luz del sol a luna y conmuta el encendido automático de las 34 farolas.
F	Faros del Auto	Enciende o apaga los haces de luz delanteros del vehículo proyectados sobre el asfalto.
M	Minimapa (Radar)	Muestra u oculta la pantalla del radar GPS en la esquina superior derecha.
P	Menú de Ayuda	Despliega una ventana superpuesta con el resumen rápido de teclas en pantalla.
R	Reiniciar	Devuelve el auto al punto de spawn, recupera los 100 HP y reconstruye objetos destruidos.
ESC	Salir	Cierra la ventana de GLFW, libera la memoria de buffers (VAO, VBO) y finaliza la ejecución.

Perspectivas de Cámara Disponibles

• Tercera Persona (Persecución): Cámara ubicada detrás y por encima del vehículo. Utiliza una interpolación lineal (lerp) para suavizar el seguimiento al doblar o frenar bruscamente.

• Primera Persona (Cabina / Capó): Fijada rígidamente en la parte delantera del auto a la altura del parabrisas, ofreciendo una sensación inmersiva de velocidad de conducción.

• Cenital Aérea (Top-Down): Cámara fija a 80 metros de altura orientada perpendicularmente hacia el suelo, ideal para inspeccionar el trazado completo de la ciudad y el movimiento del tráfico autónomo.

3. Arquitectura del Motor y Funciones Implementadas

Estructura técnica de los módulos y subsistemas implementados con Java, LWJGL 3 y OpenGL 3.3 Core Profile:

3.1 Pipeline Gráfico y Shaders GLSL (OpenGL 3.3 Core Profile)

El renderizado utiliza OpenGL 3.3 en modo moderno (Core Profile). La geometría se envía a la GPU mediante Vertex Array Objects (VAO) y Vertex Buffer Objects (VBO), procesados por dos shaders en GLSL:

- Vertex Shader: Recibe coordenadas de vértices y normales de superficie. Multiplica los vectores por las matrices de Modelo, Vista y Proyección (MVP) pasadas por uniforms y envía la posición final a la GPU.
- Fragment Shader: Implementa iluminación Phong (ambiental, difusa y especular). Calcula la luz direccional principal (sol de día, luna de noche), evalúa hasta 32 farolas puntuales activas y proyecta los dos focos delanteros del vehículo.
- Clase Pincel: Funciona como una máquina de estados con pila de matrices (traslación, rotación, escalado) que permite armar primitivas 3D limpias (cubos, cilindros, prismas) sin repetir llamadas crudas de OpenGL.

3.2 Físicas del Automóvil y Sistema de Daño (Auto.java)

El auto utiliza un modelo dinámico propio basado en vectores de aceleración, inercia, frenado y fricción contra el asfalto. El ángulo de giro de las ruedas se ajusta a la velocidad: a baja velocidad maniobra ágilmente en esquinas y a alta velocidad se vuelve más firme para no perder el control.

El vehículo tiene 100 HP de salud. Al chocar contra muros o mobiliario a velocidad se descuenta vida según la fuerza del golpe y el auto frena bruscamente. Al llegar a 0 HP, el motor se apaga y se genera humo negro mediante el emisor de partículas (Particula.java).

3.3 Objetos Destruibles con Desaparición Limpia (ObjetoDestructible.java)

Las farolas, semáforos, bancos, vallas e hidrantes heredan de ObjetoDestructible. Al ser embestidos:

- Empuje horizontal: El objeto recibe impulso en la dirección de avance del auto, desplazándose sobre el suelo.
- Animación de desintegración (0.50 s): Rota sobre su eje y reduce su tamaño a cero mientras expulsa partículas acordes a su material.
- Desactivación: Tras medio segundo se apaga del renderizado y de las colisiones para no bloquear el paso en la calle.
- Restauración: Con la tecla R todos los elementos rotos vuelven a su posición íntegra original.

3.4 Árboles con Detección de Colisión (Arbol.java)

Ubicados dentro de los 4 parques urbanos. Cuentan con colisionador cilíndrico y heredan de ObjetoDestructible. Al chocar contra uno, restan 12 HP al vehículo, expulsan 40 partículas de hojas y astillas, y desaparecen tras su animación.

3.5 Tráfico Vehicular Autónomo (AutoNPC.java)

Vehículos secundarios que circulan por las calles a velocidad constante siguiendo una lista circular de waypoints en su carril derecho. Cuentan con cajas AABB para detectar impactos contra el auto del jugador.

3.6 Sistema de Misiones de Entrega (SistemaEntregas.java)

Ciclo de 3 entregas consecutivas con destinos variables. Cada punto objetivo se marca con un haz de luz vertical en el escenario 3D y en el minimapa. Al frenar dentro del radio por 1 segundo, se cobra la entrega y se habilita el siguiente destino.

3.7 Interfaz de Usuario en Pantalla (HUD y Radar en PanelUI.java)

Renderizado 2D proyectado directamente sobre pantalla desactivando el test de profundidad:

- Velocímetro Digital: Muestra la velocidad en km/h y una barra circular indicadora.
- Barra de Salud: Indicador de 100 HP con variación de color (verde, naranja, rojo).
- Minimapa (Radar GPS): Mapa cenital con la traza de calles, posición y rotación del auto, tráfico NPC y destino de entrega.
- Panel de Estado: Muestra el sector urbano actual, luces del auto y fase horaria (día/noche).

4. Limitaciones del Proyecto

Aspectos técnicos que quedaron simplificados o fuera del alcance de esta versión:

- Modelos geométricos procedurales (sin archivos OBJ): Los edificios y vehículos se arman combinando cubos y cilindros con la clase Pincel. No se implementó un cargador de mallas externas para mantener el código autocontenido y liviano.
- Física vehicular simplificada: El manejo responde a aceleración, inercia y fricción sobre el asfalto pensado para teclado. No contempla simulación de suspensión independiente ni deformación de carrocería ante impactos.
- Colisiones mediante volúmenes envolventes (AABB): Los choques contra muros y edificios se evalúan mediante cajas de colisión alineadas con los ejes y radios de proximidad, en lugar de pruebas polígono a polígono.
- Tráfico con rutas predefinidas: Los autos secundarios circulan sobre listas fijas de waypoints por carril. Si detectan al jugador frenan para no traspasarlo, pero no recalculan rutas dinámicas ni buscan desvíos.

5. Instrucciones de Compilación y Ejecución

Comandos de Maven para compilar, probar y ejecutar el proyecto (requiere JDK 17+):

Paso	Comando de Terminal	Acción
Compilar	`mvn clean compile`	Limpia la carpeta target/ y compila las clases Java.
Pruebas	`mvn test`	Corre las pruebas unitarias de físicas, colisiones y límites de mapa.
Ejecutar	`mvn compile exec:exec`	Inicia el simulador abriendo la ventana gráfica de OpenGL.
Empaquetar	`mvn package`	Genera el archivo JAR ejecutable en la carpeta target/.

6. Librerías y Dependencias (pom.xml)

Dependencias externas configuradas en el proyecto:

Librería	Versión	Uso en el Proyecto
LWJGL 3	3.3.3	Bindings nativos para la ventana GLFW, eventos de teclado y llamadas a OpenGL 3.3.
JOML	1.10.5	Álgebra lineal para gráficos: matrices 4x4 (MVP), vectores 3D y transformaciones.
JUnit	4.13.2	Framework para pruebas unitarias automatizadas de la lógica sin ventana gráfica.